K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

DSA LAB CODES

COURSE CODE - ENCS256

NAME : Vipul

COURSE : B.TECH CSE(FSD)

ROLL NO : 2401350017

GITHUB LINK :LINK

1. Factorial and Fibonacci :

Implement the following recursive algorithms:

1. Factorial
2. Fibonacci (Naïve Recursive)
3. Fibonacci (Dynamic Programming)

Steps

- Count number of function calls for each input size.
- Measure execution time.
- Compare recursive vs optimized approach.

Deliverables

- Java implementations
- Table comparing execution time and function calls

CODE:

```
import java.util.HashMap;



public class RecursionAnalysis {



    static int countRec = 0;

    static int countDP = 0;



    // Simple recursive Fibonacci

    static int fib(int n) {

        countRec++;



        if (n == 0)

            return 0;
```

```
        if (n == 1)
            return 1;

        return fib(n - 1) + fib(n - 2);
    }

    // Fibonacci using DP (memoization)
    static int fibDP(int n, HashMap<Integer, Integer> map) {
        countDP++;

        if (n == 0)
            return 0;

        if (n == 1)
            return 1;

        if (map.containsKey(n)) {
            return map.get(n);
        }

        int ans = fibDP(n - 1, map) + fibDP(n - 2, map);

        map.put(n, ans);

        return ans;
```

```
    }

    public static void main(String[] args) {

        int arr[] = {5, 10, 20, 30};

        System.out.println("Time Comparison:");
        System.out.println("n  Recursion(ms)  DP(ms)");

        for (int i = 0; i < arr.length; i++) {

            int n = arr[i];

            countRec = 0;
            countDP = 0;

            long start = System.nanoTime();
            fib(n);
            long end = System.nanoTime();

            double time1 = (end - start) / 1000000.0;

            start = System.nanoTime();
            fibDP(n, new HashMap<>());
            end = System.nanoTime();
```

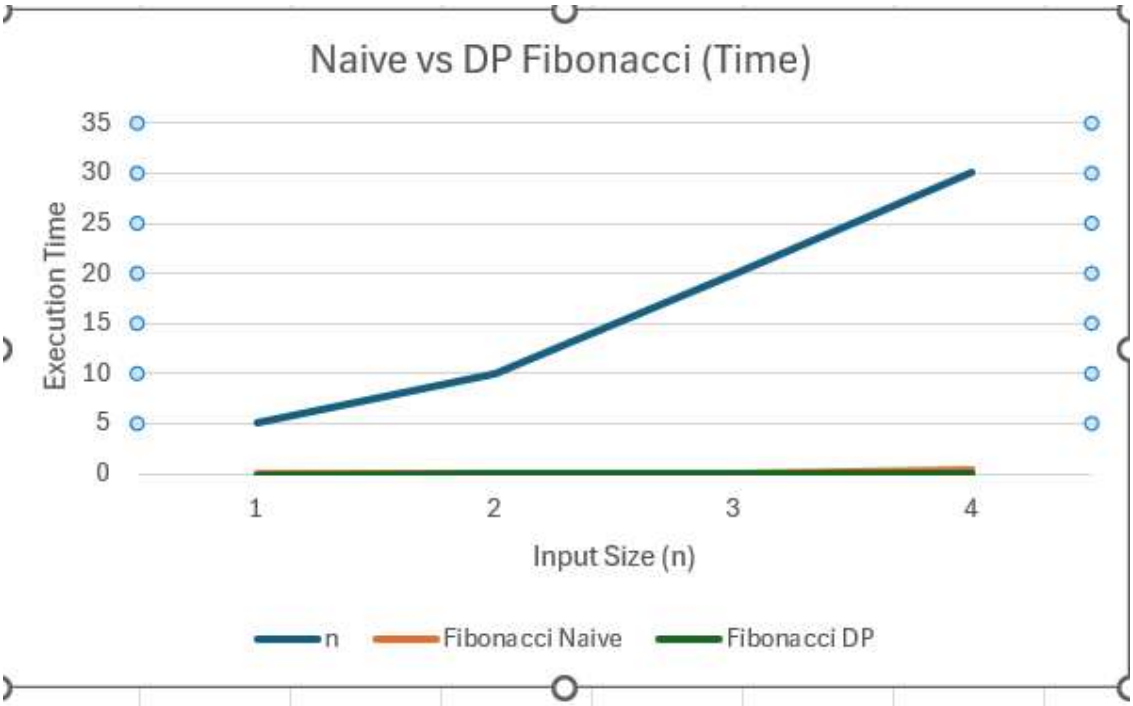
```
        double time2 = (end - start) / 1000000.0;


        System.out.println(n + "   " + time1 + "   " + time2);
    }


    System.out.println("\nFunction Calls:");
    System.out.println("n  RecCalls  DPCalls");


    for (int i = 0; i < arr.length; i++) {


        int n = arr[i];


        countRec = 0;
        countDP = 0;


        fib(n);
        int r = countRec;


        fibDP(n, new HashMap<>());
        int d = countDP;


        System.out.println(n + "   " + r + "   " + d);
    }
}
```

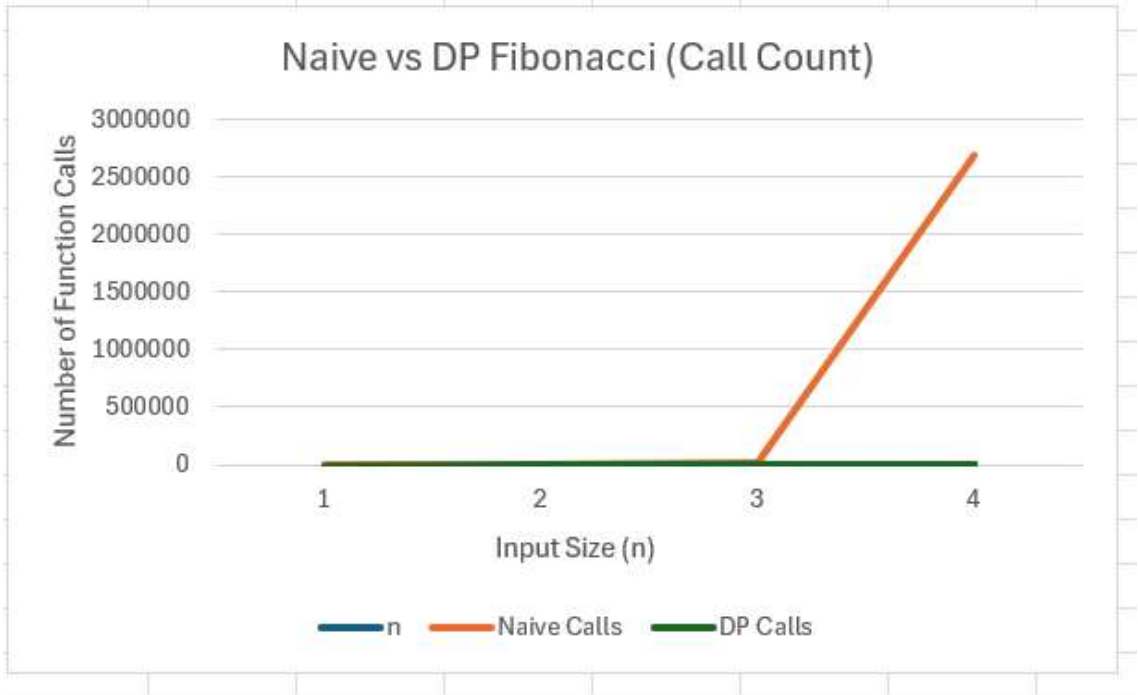
```
}
```

Observations

1. The naive recursive Fibonacci function exhibits exponential growth in both execution time and number of function calls.
2. For larger inputs such as $n = 30$, the number of recursive calls grows into the millions, clearly reflecting its **$O(2^n)$** time complexity.
3. In contrast, the dynamic programming approach (memoization) significantly reduces redundant computations by storing previously computed values.
4. As a result, the number of function calls and execution time increase almost linearly with input size, demonstrating an **O(n)** time complexity.
5. The experimental results, obtained through measured execution time and call count analysis, clearly highlight the efficiency gained using optimization techniques like memoization

GRAPHS:


Naive vs DP Fibonacci (Time)
Execution Time
35
30
25
20
15
10
5
0
1
2
3
4
Input Size (n)
n
Fibonacci Naive
Fibonacci DP


Naive vs DP Fibonacci (Call Count)
Number of Function Calls
3000000
2500000
2000000
1500000
1000000
500000
0
1
2
3
4
Input Size (n)
n
Naive Calls
DP Calls

2. Sorting:

Sorting a Set of Numbers Using Various Algorithms and Analyzing Their Efficiency .

Introduction :

Sorting algorithms are fundamental in computer science, used to arrange data in a specific order to enhance efficiency and accessibility. This assignment focuses on implementing various sorting techniques to organize numbers into ascending or descending order. By utilizing the step-count method, we will calculate the time complexity of each algorithm to analyze their performance. Input data will be taken from a table and operations will be repeated with varying input sizes (10, 20, 30, 40 numbers). The study includes examining best-case, worst-case, and average-case scenarios, culminating in visualizing results through graphs to interpret computational trends effectively.

Objective:

The objective of this assignment is to implement multiple sorting algorithms, sort a given set of numbers in ascending and descending order, and analyze their performance using the step-count method. Additionally, students will examine the best-case, worst-case, and average-case scenarios and visualize the results through graphs.

CODE:

```
import java.util.Random;

public class SortingAnalysis {

    // swap function
    static void swap(int arr[], int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    // Bubble Sort
    static int bubbleSort(int arr[]) {
        int count = 0;

        for (int i = 0; i < arr.length - 1; i++) {

            for (int j = 0; j < arr.length - i - 1; j++) {
                count++;

                if (arr[j] > arr[j + 1]) {
                    swap(arr, j, j + 1);
                }
            }
        }
        return count;
    }

    // Selection Sort
    static int selectionSort(int arr[]) {
        int count = 0;

        for (int i = 0; i < arr.length - 1; i++) {

            int min = i;

            for (int j = i + 1; j < arr.length; j++) {
```

```
                count++;

                if (arr[j] < arr[min]) {
                    min = j;
                }
            }

            swap(arr, i, min);
        }
        return count;
    }

    // Insertion Sort
    static int insertionSort(int arr[]) {
        int count = 0;

        for (int i = 1; i < arr.length; i++) {

            int key = arr[i];
            int j = i - 1;

            while (j >= 0 && arr[j] > key) {
                count++;
                arr[j + 1] = arr[j];
                j--;
            }

            count++; // for last comparison
            arr[j + 1] = key;
        }

        return count;
    }

    // Random array
    static int[] randomArray(int n) {
        Random r = new Random();
        int arr[] = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = r.nextInt(100);
        }
```

```
        return arr;
    }

    // Sorted array
    static int[] sortedArray(int n) {
        int arr[] = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = i;
        }

        return arr;
    }

    // Reverse array
    static int[] reverseArray(int n) {
        int arr[] = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = n - i;
        }

        return arr;
    }

    // print results
    static void show(String type, int n, int arr[]) {

        int a[] = arr.clone();
        int b[] = arr.clone();
        int c[] = arr.clone();

        System.out.println("\n" + type + " case (n=" + n + ")");

        System.out.println("Bubble: " + bubbleSort(a));
        System.out.println("Selection: " + selectionSort(b));
        System.out.println("Insertion: " + insertionSort(c));
    }

    public static void main(String[] args) {

        int size[] = {10, 20, 30};
```

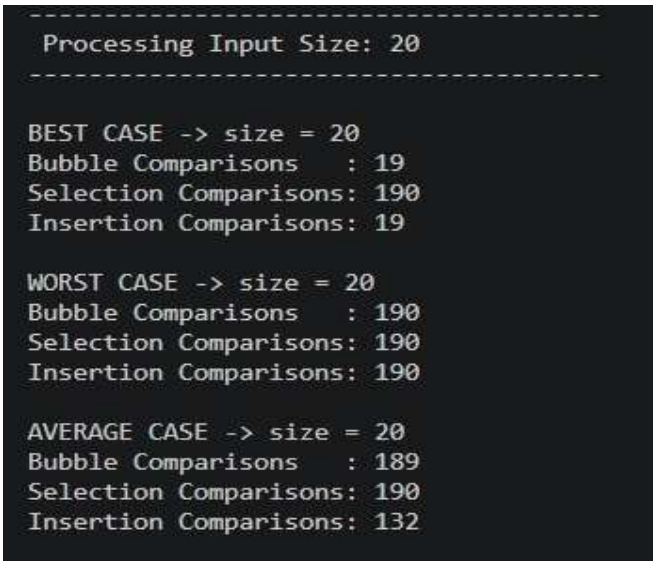
```
        for (int i = 0; i < size.length; i++) {

            int n = size[i];

            System.out.println("\n----- n = " + n + " -----");

            show("Best", n, sortedArray(n));
            show("Worst", n, reverseArray(n));
            show("Average", n, randomArray(n));
        }
    }
}
```

OUTPUT:

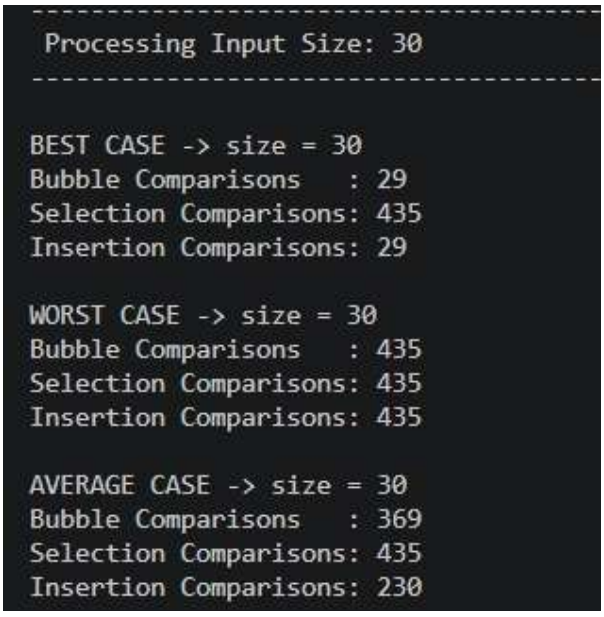
FOR INPUT SIZE 10:

```
---------------------------------------
 Processing Input Size: 10
---------------------------------------

BEST CASE -> size = 10
Bubble Comparisons   : 9
Selection Comparisons: 45
Insertion Comparisons: 9

WORST CASE -> size = 10
Bubble Comparisons   : 45
Selection Comparisons: 45
Insertion Comparisons: 45

AVERAGE CASE -> size = 10
Bubble Comparisons   : 45
Selection Comparisons: 45
Insertion Comparisons: 36
```

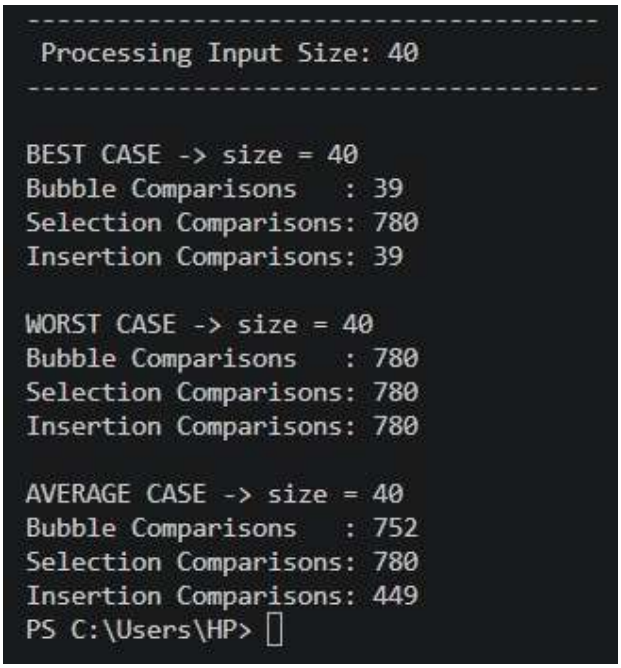
FOR INPUT SIZE 20:



FOR INPUT SIZE 30:



FOR INPUT SIZE 40:

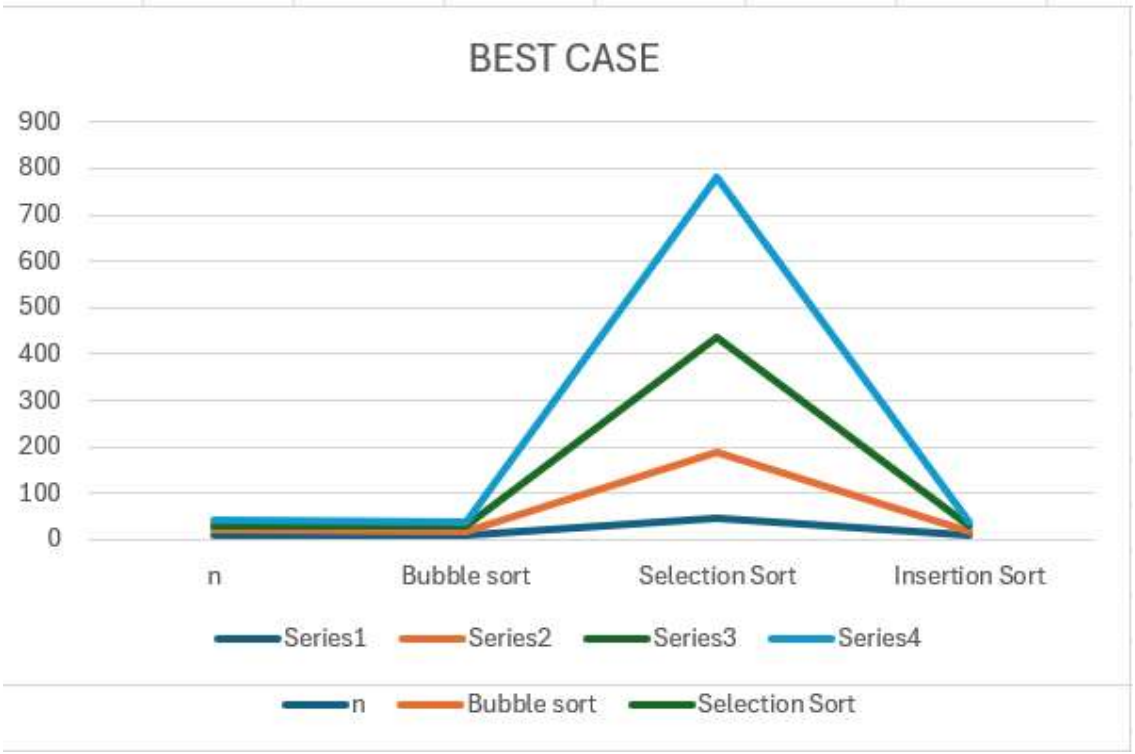


TABLES:

BEST CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	9	45	9
20	19	190	19
30	29	435	29
40	39	780	39

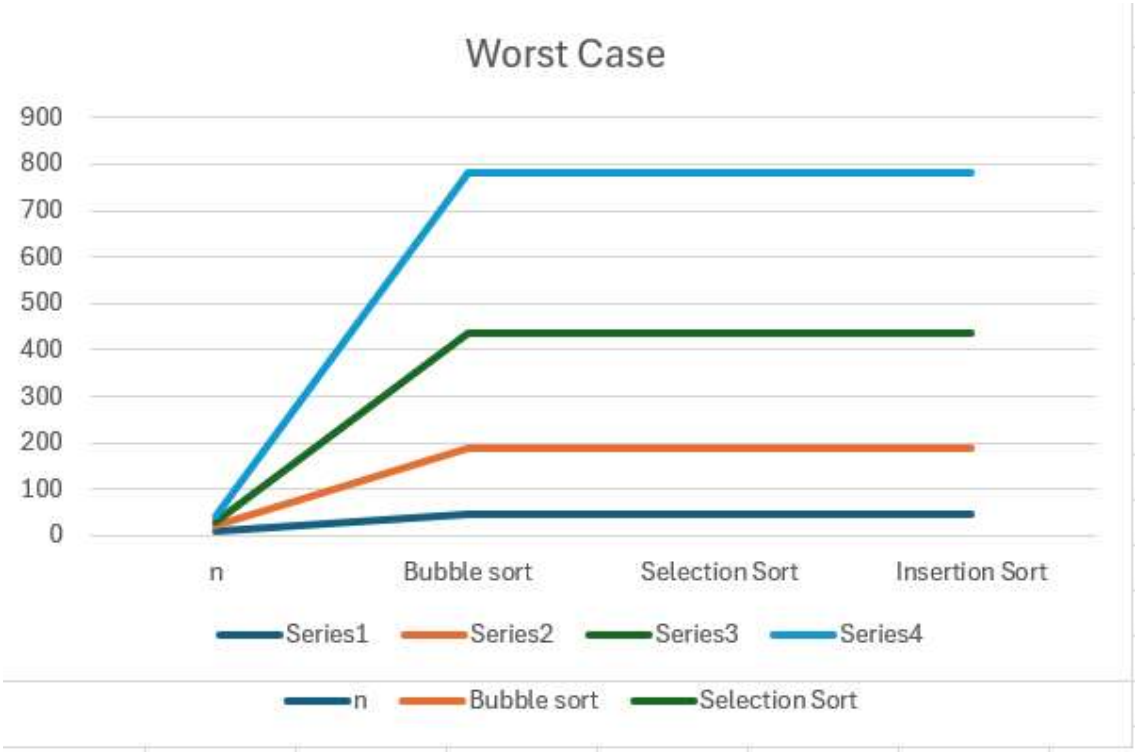
GRAPH



WORST CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	45	45	45
20	190	190	190
30	435	435	435
40	780	780	780

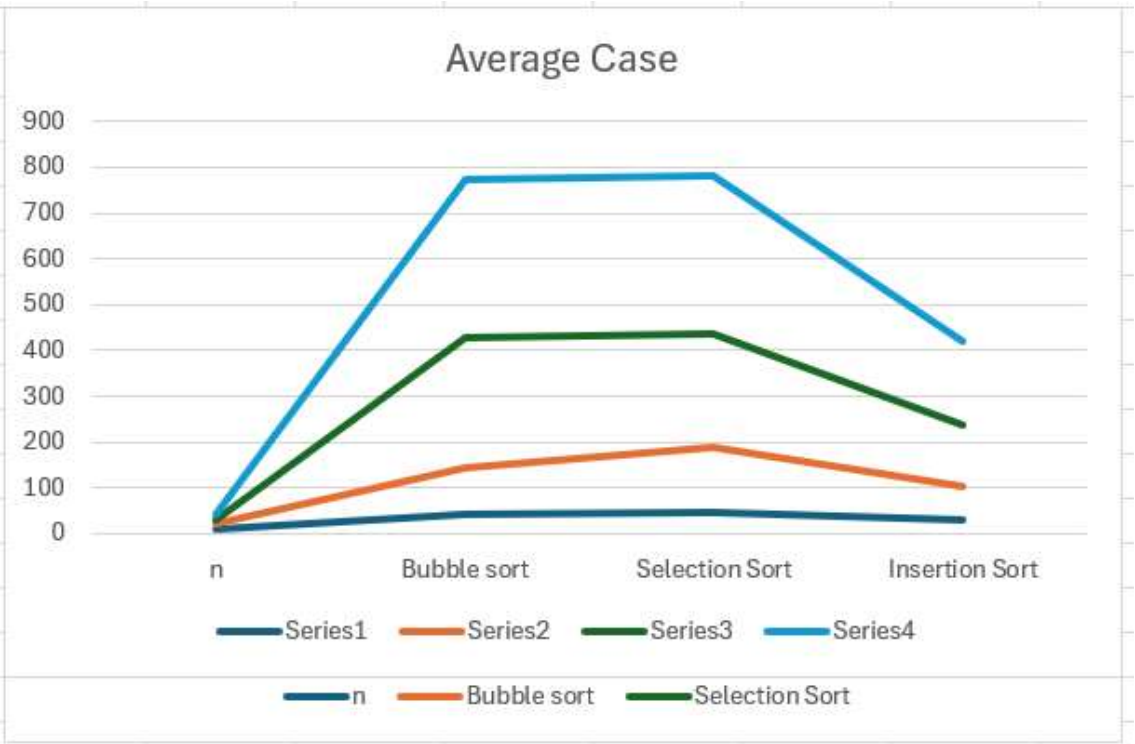
GRAPH



AVERAGE CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	44	45	30
20	145	190	104
30	429	435	236
40	774	780	421

GRAPH



Observation From Graphs:

From the graphs and comparison count tables, we observe that:

1. **Selection Sort** shows almost the same number of comparisons in best, average, and worst cases.
   This is because it always scans the entire unsorted portion of the array regardless of input order.
   Hence, its time complexity remains **$O(n^2)$** in all cases.

2. **Bubble Sort** performs very efficiently in the best case due to the optimization (early stopping when no swaps occur).
   Therefore, its best-case complexity becomes **$O(n)$**.

However, in average and worst cases, it performs nearly **$O(n^2)$** comparisons.

3. **Insertion Sort** performs efficiently for already sorted data, showing linear growth in the best case (**$O(n)$**).
   In the worst case, it performs **$O(n^2)$** comparisons, similar to Bubble and Selection Sort.

4. From the average-case graph, Insertion Sort performs better than Bubble Sort for larger inputs.

3. Searching:

Implement Linear Search and Binary Search.

Requirements

- Generate random arrays of different sizes.
- Measure execution time for:
- Best case
- Average case
- Worst case
- Compare results using plots.

Deliverables

- Search algorithm implementations
- Comparison plots
- Observations in comments/markdown

CODE :

```
import java.util.*;



public class SearchAnalysis {



    // Linear Search

    static int linear(int arr[], int key) {



        for (int i = 0; i < arr.length; i++) {

            if (arr[i] == key) {

                return i;

            }

        }
```

```
        return -1;
    }

    // Binary Search
    static int binary(int arr[], int key) {

        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {

            int mid = (low + high) / 2;

            if (arr[mid] == key) {
                return mid;
            }

            if (arr[mid] < key) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
```

```
        return -1;
    }

    // Create sorted array
    static int[] makeArray(int n) {

        Random r = new Random();
        int arr[] = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = r.nextInt(100);
        }

        Arrays.sort(arr);
        return arr;
    }

    public static void main(String[] args) {

        int size[] = {10, 100, 500};
        int repeat = 500;

        System.out.println("Worst Case:");
        System.out.println("n   Linear   Binary");
```

```
        for (int i = 0; i < size.length; i++) {


            int n = size[i];

            int arr[] = makeArray(n);

            int key = -1; // not present


            long start = System.nanoTime();

            for (int j = 0; j < repeat; j++) {

                linear(arr, key);

            }

            long end = System.nanoTime();

            double t1 = (end - start) / 1000000.0;


            start = System.nanoTime();

            for (int j = 0; j < repeat; j++) {

                binary(arr, key);

            }

            end = System.nanoTime();

            double t2 = (end - start) / 1000000.0;


            System.out.println(n + "    " + t1 + "    " + t2);

        }


        System.out.println("\nAverage Case:");

        System.out.println("n    Linear    Binary");
```

```
for (int i = 0; i < size.length; i++) {


    int n = size[i];

    int arr[] = makeArray(n);

    int key = arr[n / 2];


    long start = System.nanoTime();

    for (int j = 0; j < repeat; j++) {

        linear(arr, key);

    }

    long end = System.nanoTime();

    double t1 = (end - start) / 1000000.0;


    start = System.nanoTime();

    for (int j = 0; j < repeat; j++) {

        binary(arr, key);

    }

    end = System.nanoTime();

    double t2 = (end - start) / 1000000.0;


    System.out.println(n + "    " + t1 + "    " + t2);

}


System.out.println("\nBest Case:");
```

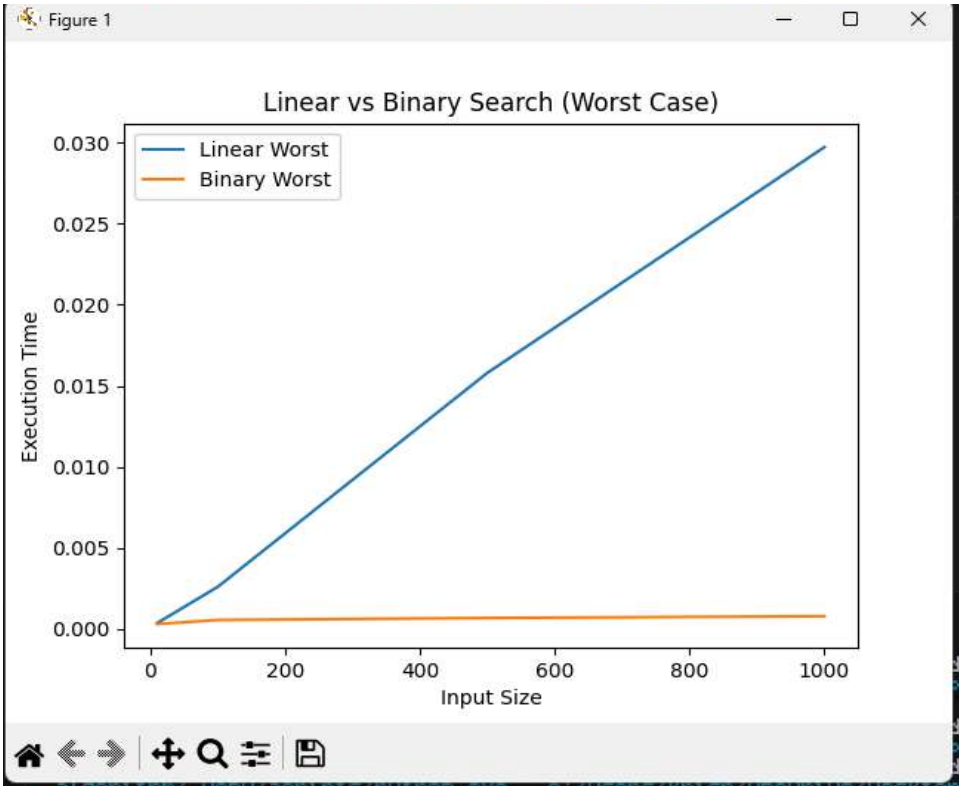
```
        System.out.println("n   Linear   Binary");


        for (int i = 0; i < size.length; i++) {


            int n = size[i];

            int arr[] = makeArray(n);

            int key = arr[0];


            long start = System.nanoTime();

            for (int j = 0; j < repeat; j++) {

                linear(arr, key);

            }

            long end = System.nanoTime();

            double t1 = (end - start) / 1000000.0;


            start = System.nanoTime();

            for (int j = 0; j < repeat; j++) {

                binary(arr, key);

            }

            end = System.nanoTime();

            double t2 = (end - start) / 1000000.0;


            System.out.println(n + "   " + t1 + "   " + t2);

        }

    }}
```

Conclusion :

Binary search is significantly more efficient than linear search for large input sizes.
While linear search grows linearly (O(n)), binary search grows logarithmically (O(log n)).
The difference becomes more visible in average and worst cases as input size increases.

Graphs:

WORST CASE



AVERAGE CASE

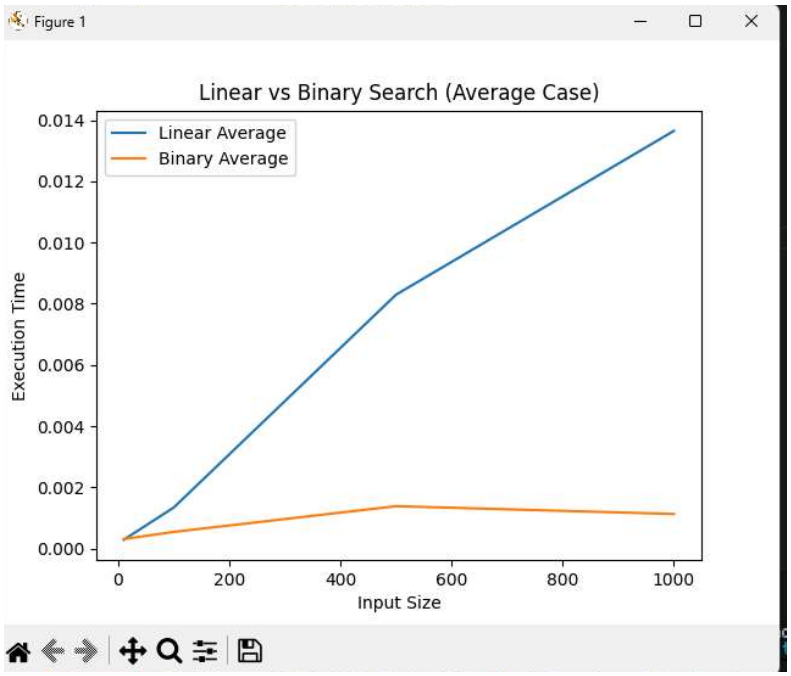


Figure 1
Linear vs Binary Search (Average Case)
Linear Average
Binary Average
Execution Time
0.014
0.012
0.010
0.008
0.006
0.004
0.002
0.000
0
200
400
600
800
1000
Input Size

BEST CASE

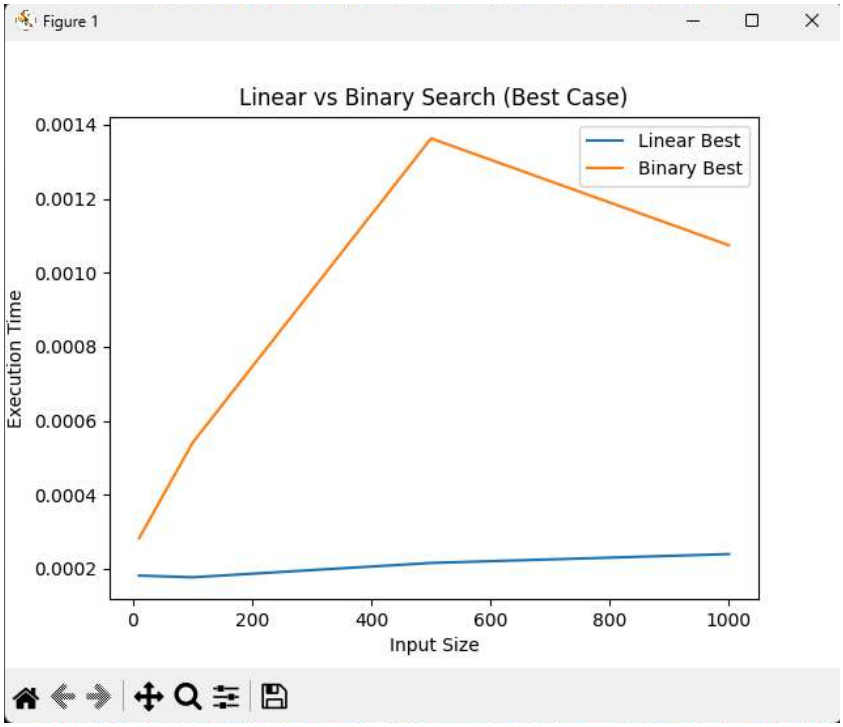


Figure 1
Linear vs Binary Search (Best Case)
Linear Best
Binary Best
Execution Time
0.0014
0.0012
0.0010
0.0008
0.0006
0.0004
0.0002
0
200
400
600
800
1000
Input Size

4. **Merge Sort and Quick Sort:**

~ Merge Sort:

Aim:

To implement Merge Sort using divide and conquer approach.

Theory:

Merge Sort divides the array into smaller parts, sorts them recursively and merges them.

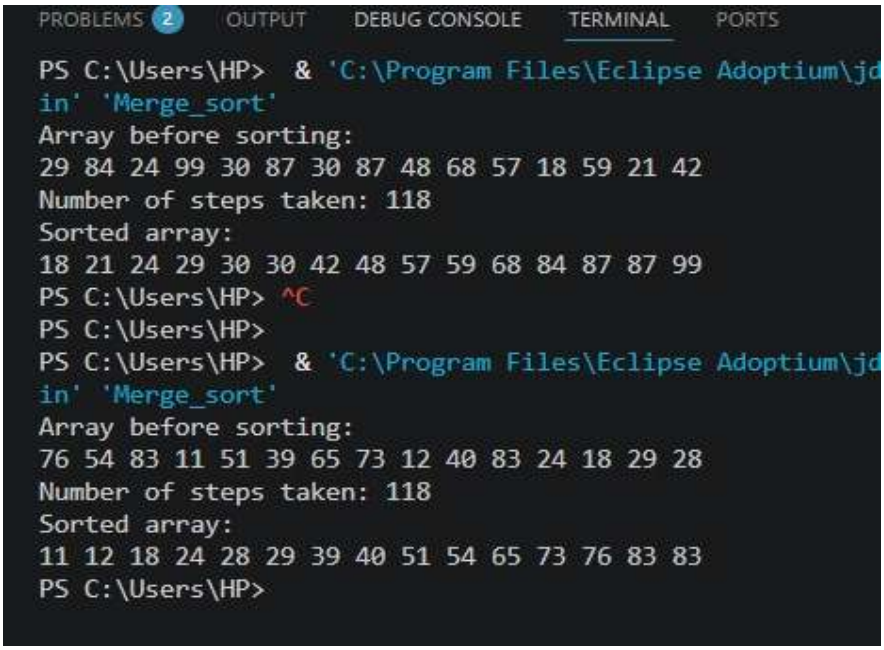
Code :

```
public class MergeSort {

    public static void main(String[] args) {

        int arr[] = new int[10];

        // fill random values
        for (int i = 0; i < arr.length; i++) {
            arr[i] = (int)(Math.random() * 100);
        }

        System.out.println("Before sorting:");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }

        int steps = mergeSort(arr);

        System.out.println("\nSteps: " + steps);

        System.out.println("After sorting:");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
    }
```

```
    // merge sort
    static int mergeSort(int arr[]) {

        int count = 0;

        if (arr.length <= 1) {
            return count;
        }

        int mid = arr.length / 2;

        int left[] = new int[mid];
        int right[] = new int[arr.length - mid];

        // copy left
        for (int i = 0; i < mid; i++) {
            left[i] = arr[i];
            count++;
        }

        // copy right
        for (int i = mid; i < arr.length; i++) {
            right[i - mid] = arr[i];
            count++;
        }

        count += mergeSort(left);
        count += mergeSort(right);

        count += merge(arr, left, right);

        return count;
    }

    // merge function
    static int merge(int arr[], int left[], int right[]) {

        int i = 0, j = 0, k = 0;
        int count = 0;

        while (i < left.length && j < right.length) {

            if (left[i] < right[j]) {
```

```
                arr[k] = left[i];
                i++;
            } else {
                arr[k] = right[j];
                j++;
            }

            k++;
            count++;
        }

        while (i < left.length) {
            arr[k] = left[i];
            i++;
            k++;
            count++;
        }

        while (j < right.length) {
            arr[k] = right[j];
            j++;
            k++;
            count++;
        }

        return count;
    }
}
```

Output:



~ Quick Sort:

Aim:

To implement Quick Sort and analyze its performance.

Theory:

Quick Sort uses a pivot element to partition the array and recursively sorts subarrays.

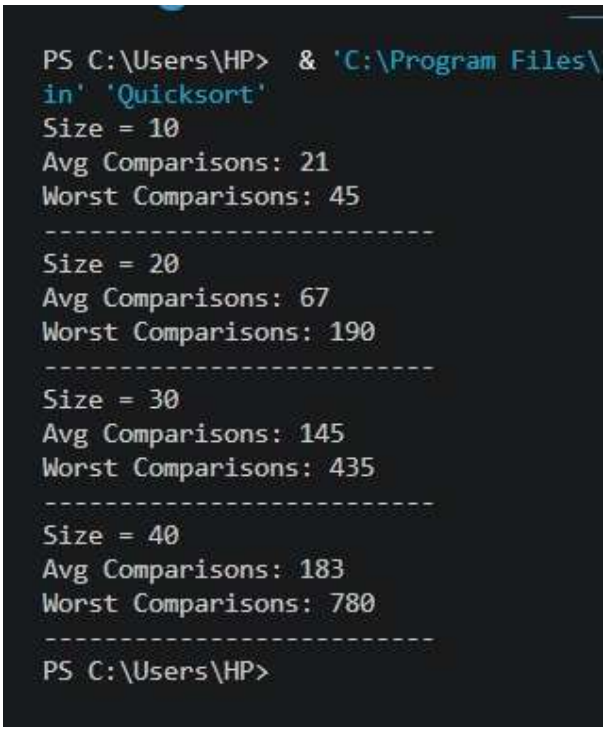
Code :

```
import java.util.Random;

public class QuickSort {

    static int count = 0;

    // partition
    static int partition(int arr[], int low, int high) {

        int pivot = arr[high];
```

```
        int i = low - 1;

        for (int j = low; j < high; j++) {

            count++;

            if (arr[j] < pivot) {
                i++;

                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;

        return i + 1;
    }

    // quicksort
    static void quick(int arr[], int low, int high) {

        if (low < high) {

            int p = partition(arr, low, high);

            quick(arr, low, p - 1);
            quick(arr, p + 1, high);
        }
    }

    // random array
    static int[] randomArray(int n) {

        Random r = new Random();
        int arr[] = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = r.nextInt(100);
        }
```

```
        return arr;
    }

    // sorted array (worst case)
    static int[] sortedArray(int n) {

        int arr[] = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = i;
        }

        return arr;
    }

    public static void main(String[] args) {

        int size[] = {10, 20, 30};

        for (int i = 0; i < size.length; i++) {

            int n = size[i];

            System.out.println("\nSize = " + n);

            // average case
            int a[] = randomArray(n);
            count = 0;
            quick(a, 0, n - 1);
            System.out.println("Average: " + count);

            // worst case
            int b[] = sortedArray(n);
            count = 0;
            quick(b, 0, n - 1);
            System.out.println("Worst: " + count);
        }
    }
}
```

Output:



```
PS C:\Users\HP>  & 'C:\Program Files\
in' 'Quicksort'
Size = 10
Avg Comparisons: 21
Worst Comparisons: 45
--------------------------
Size = 20
Avg Comparisons: 67
Worst Comparisons: 190
--------------------------
Size = 30
Avg Comparisons: 145
Worst Comparisons: 435
--------------------------
Size = 40
Avg Comparisons: 183
Worst Comparisons: 780
--------------------------
PS C:\Users\HP>
```

5. Fractional Knapsack

Aim:

To maximize profit using Fractional Knapsack.

Theory:

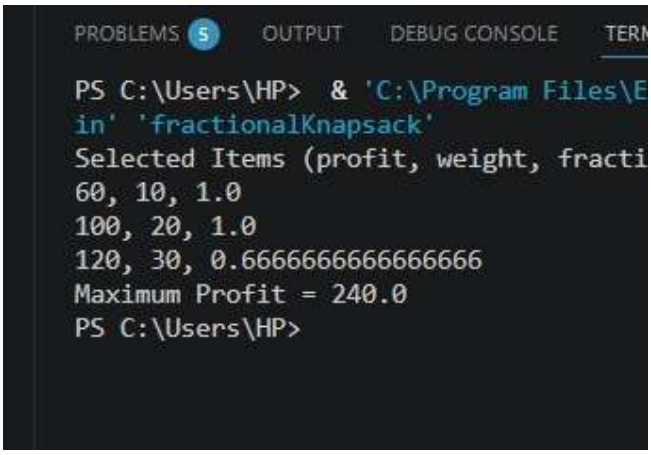
Items can be taken partially. Greedy approach based on value/weight ratio is used.

Code:

```
import java.util.*;

class Item {
    int profit;
    int weight;
    double ratio;

    Item(int p, int w) {
        profit = p;
        weight = w;
        ratio = (double)p / w;
    }
}

public class Knapsack {

    static void solve(Item arr[], int cap) {

        // sort based on ratio (high to low)
        Arrays.sort(arr, (a, b) -> {
            if (b.ratio > a.ratio)
                return 1;
            else
                return -1;
        });

        double total = 0;

        System.out.println("Items taken:");
```

```
        for (int i = 0; i < arr.length; i++) {

            if (cap >= arr[i].weight) {

                cap = cap - arr[i].weight;
                total = total + arr[i].profit;

                System.out.println(arr[i].profit + " " + arr[i].weight
+ " full");

            } else {

                double frac = (double)cap / arr[i].weight;

                total = total + arr[i].profit * frac;

                System.out.println(arr[i].profit + " " + arr[i].weight
+ " " + frac);

                break;
            }
        }

        System.out.println("Max Profit = " + total);
    }

    public static void main(String[] args) {

        Item arr[] = {
            new Item(60, 10),
            new Item(100, 20),
            new Item(120, 30),
            new Item(80, 25)
        };

        int cap = 50;

        solve(arr, cap);
    }
}
```

Output:


PROBLEMS 5
OUTPUT
DEBUG CONSOLE
TERM
PS C:\Users\HP> & 'C:\Program Files\E
in' 'fractionalKnapsack'
Selected Items (profit, weight, fracti
60, 10, 1.0
100, 20, 1.0
120, 30, 0.6666666666666666
Maximum Profit = 240.0
PS C:\Users\HP>

6. 0/1 KNAPSACK

Aim:

To solve 0/1 Knapsack using dynamic programming.

Theory:

Items can either be taken or not. DP is used to maximize profit.

CODES:

```
public class Knapsack01 {

    static void solve(int wt[], int val[], int n, int cap) {

        int dp[][] = new int[n + 1][cap + 1];

        // fill table
        for (int i = 0; i <= n; i++) {

            for (int j = 0; j <= cap; j++) {

                if (i == 0 || j == 0) {
                    dp[i][j] = 0;
                }

                else if (wt[i - 1] <= j) {
                    dp[i][j] = Math.max(
                            val[i - 1] + dp[i - 1][j - wt[i - 1]],
                            dp[i - 1][j]
                    );
                }

                else {
                    dp[i][j] = dp[i - 1][j];
                }
            }
        }

        System.out.println("Max Profit = " + dp[n][cap]);
```

```
        // find selected items
        int j = cap;

        System.out.println("Items selected:");

        for (int i = n; i > 0 && j > 0; i--) {

            if (dp[i][j] != dp[i - 1][j]) {
                System.out.println("Item " + i + " taken");
                j = j - wt[i - 1];
            } else {
                System.out.println("Item " + i + " not taken");
            }
        }
    }

    public static void main(String[] args) {

        int val[] = {40, 70, 80, 120};
        int wt[] = {5, 10, 20, 30};

        int cap = 40;
        int n = val.length;

        solve(wt, val, n, cap);
    }
}
```

Output :

```
PS C:\Users\HP> & 'C:\Program Files\
'C:\Users\HP\AppData\Roaming\Code\User\workspaceStorage\ce64bce2bbaeb9479d78
Best Profit = 190
Chosen Items (0/1):
Product 4 -> 0
Product 3 -> 1
Product 2 -> 1
Product 1 -> 1
PS C:\Users\HP>
```

7. Longest common subsequence (LCS):

Aim:

To find longest common subsequence between two strings.

Theory:

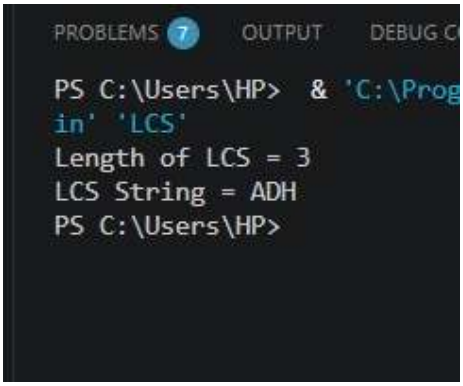
LCS finds common elements in same order using DP.

Code:

```
public class LCS {

    static void findLCS(String s1, String s2) {

        int n = s1.length();
        int m = s2.length();

        int dp[][] = new int[n + 1][m + 1];

        // fill dp table
        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= m; j++) {

                if (s1.charAt(i - 1) == s2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                } else {
                    if (dp[i - 1][j] > dp[i][j - 1])
                        dp[i][j] = dp[i - 1][j];
                    else
                        dp[i][j] = dp[i][j - 1];
                }
            }
        }

        System.out.println("LCS length = " + dp[n][m]);

        // find LCS string
        int i = n;
        int j = m;
```

```
        String ans = "";

        while (i > 0 && j > 0) {

            if (s1.charAt(i - 1) == s2.charAt(j - 1)) {
                ans = s1.charAt(i - 1) + ans;
                i--;
                j--;
            }
            else if (dp[i - 1][j] > dp[i][j - 1]) {
                i--;
            }
            else {
                j--;
            }
        }

        System.out.println("LCS = " + ans);
    }

    public static void main(String[] args) {

        String a = "ABCDGH";
        String b = "AEDFHR";

        findLCS(a, b);
    }
}
```

Output :



8. Matrix Chain Multiplication:

Aim:

To find minimum scalar multiplications in matrix chain.

Theory:

Optimal parenthesis reduces multiplication cost.

Code :

```
public class MCM {

    static void solve(int p[], int n) {

        int dp[][] = new int[n][n];
        int bracket[][] = new int[n][n];

        // cost is 0 when multiplying one matrix
        for (int i = 1; i < n; i++) {
            dp[i][i] = 0;
        }

        // length of chain
        for (int len = 2; len < n; len++) {

            for (int i = 1; i < n - len + 1; i++) {

                int j = i + len - 1;
                dp[i][j] = Integer.MAX_VALUE;

                for (int k = i; k < j; k++) {

                    int cost = dp[i][k] + dp[k + 1][j]
                            + p[i - 1] * p[k] * p[j];

                    if (cost < dp[i][j]) {
                        dp[i][j] = cost;
                        bracket[i][j] = k;
                    }
                }
```

```
            }
        }

        System.out.println("Minimum multiplications = " + dp[1][n -
1]);

        System.out.print("Order: ");
        print(bracket, 1, n - 1);
    }

    static void print(int bracket[][], int i, int j) {

        if (i == j) {
            System.out.print((char)('A' + i - 1));
            return;
        }

        System.out.print("(");
        print(bracket, i, bracket[i][j]);
        print(bracket, bracket[i][j] + 1, j);
        System.out.print(")");
    }

    public static void main(String[] args) {

        int arr[] = {6, 11, 3, 12, 5};

        int n = arr.length;

        solve(arr, n);
    }
}
```

Output :

```
PS C:\Users\HP> & 'C:\Program Files\eclipse\jdk-17\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\HP\AppData\Local\Temp\cp_5b175e91712e4501741e4a525\redhat.java\
jdt_ws\dynamicProgramming_43c6121c\bin'
Matrices:
A = 6 x 11
B = 11 x 3
C = 3 x 12
D = 12 x 5
E = 5 x 51
F = 51 x 7

Minimum Multiplications = 2340
Optimal Parenthesization: ((AB)(((CD)E)F))

PS C:\Users\HP>
```

9. Dijkstra

Aim:

To find shortest path using Dijkstra algorithm.

Theory:

Greedy algorithm used for shortest path in graphs with non-negative weights.

Code :

```
import java.util.*;

public class Dijkstra {

    static int V = 6;

    // find min distance node
    static int minNode(int dist[], boolean visited[]) {

        int min = Integer.MAX_VALUE;
        int index = -1;

        for (int i = 0; i < V; i++) {
            if (!visited[i] && dist[i] < min) {
                min = dist[i];
                index = i;
            }
        }

        return index;
    }

    static void dijkstra(int graph[][], int src) {

        int dist[] = new int[V];
        boolean visited[] = new boolean[V];

        // initialize
        for (int i = 0; i < V; i++) {
            dist[i] = Integer.MAX_VALUE;
```

```
            visited[i] = false;
        }

        dist[src] = 0;

        for (int i = 0; i < V - 1; i++) {

            int u = minNode(dist, visited);
            visited[u] = true;

            for (int v = 0; v < V; v++) {

                if (!visited[v] && graph[u][v] != 0 &&
                    dist[u] != Integer.MAX_VALUE &&
                    dist[u] + graph[u][v] < dist[v]) {

                    dist[v] = dist[u] + graph[u][v];
                }
            }
        }

        // print result
        System.out.println("Vertex   Distance from source");

        for (int i = 0; i < V; i++) {
            System.out.println(i + "        " + dist[i]);
        }
    }

    public static void main(String[] args) {

        int graph[][] = {
            {0, 5, 1, 0, 8, 0},
            {5, 0, 6, 0, 0, 0},
            {1, 6, 0, 3, 0, 5},
            {0, 0, 3, 0, 8, 3},
            {8, 0, 0, 8, 0, 9},
            {0, 0, 5, 3, 9, 0}
        };

        int source = 0;

        dijkstra(graph, source);
```

```
    }
}
```

Output :

```
PS C:\Users\HP>  & 'C:\Program Files
in' 'Dijkstra'
Node     Shortest Distance
0                2147483647
1                1
2                7
3                10
4                18
5                12
PS C:\Users\HP>
```

10. Bellman Ford

Code :

```
public class BellmanFord {

    static class Edge {
        int u, v, w;

        Edge(int a, int b, int c) {
            u = a;
            v = b;
            w = c;
        }
    }

    static void solve(int V, Edge edges[], int src) {

        int dist[] = new int[V];

        // initialize
        for (int i = 0; i < V; i++) {
            dist[i] = Integer.MAX_VALUE;
        }

        dist[src] = 0;

        // relax edges V-1 times
        for (int i = 0; i < V - 1; i++) {
```

```
            for (int j = 0; j < edges.length; j++) {

                int u = edges[j].u;
                int v = edges[j].v;
                int w = edges[j].w;

                if (dist[u] != Integer.MAX_VALUE && dist[u] + w <
dist[v]) {
                    dist[v] = dist[u] + w;
                }
            }
        }

        // check negative cycle
        for (int j = 0; j < edges.length; j++) {

            int u = edges[j].u;
            int v = edges[j].v;
            int w = edges[j].w;

            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v])
{
                System.out.println("Negative cycle exists");
                return;
            }
        }

        // print result
        System.out.println("Vertex   Distance");

        for (int i = 0; i < V; i++) {
            System.out.println(i + "        " + dist[i]);
        }
    }

    public static void main(String[] args) {

        int V = 5;

        Edge edges[] = {
            new Edge(0, 1, 6),
            new Edge(0, 2, 7),
```

```
            new Edge(1, 2, 8),
            new Edge(1, 3, 5),
            new Edge(1, 4, -4),
            new Edge(2, 3, -3),
            new Edge(2, 4, 9),
            new Edge(3, 1, -2),
            new Edge(4, 3, 7)
        };

        int source = 0;

        solve(V, edges, source);
    }
}
```

Output:

```
PS C:\Users\HP> & 'C:\Program Files\eclipse\jdk-17\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\HP\AppData\Local\Temp\cp_5b175e91712e4501741e4a525\redhat.java\
jdt_ws\dynamicProgramming_43c6121c\bin'
Vertex   Distance from Source
0              0
1             -1
2              2
3             -2
4              1
PS C:\Users\HP>
```

11. BFS and DFS:

Aim:

To traverse graph using BFS and DFS.

Theory:

BFS uses queue, DFS uses recursion.

Code:

```
import java.util.*;

public class Graph {

    int V;
    ArrayList<Integer> adj[];

    Graph(int v) {
        V = v;

        adj = new ArrayList[V];

        // initialize list
        for (int i = 0; i < V; i++) {
            adj[i] = new ArrayList<>();
        }
    }

    // add edge
    void addEdge(int u, int v) {
        adj[u].add(v);
    }

    // BFS
    void bfs(int start) {

        boolean visited[] = new boolean[V];
        Queue<Integer> q = new LinkedList<>();
```

```
        visited[start] = true;
        q.add(start);

        System.out.print("BFS: ");

        while (!q.isEmpty()) {

            int node = q.poll();
            System.out.print(node + " ");

            for (int i = 0; i < adj[node].size(); i++) {

                int next = adj[node].get(i);

                if (!visited[next]) {
                    visited[next] = true;
                    q.add(next);
                }
            }
        }
        System.out.println();
    }

    // DFS
    void dfs(int node, boolean visited[]) {

        visited[node] = true;
        System.out.print(node + " ");

        for (int i = 0; i < adj[node].size(); i++) {

            int next = adj[node].get(i);

            if (!visited[next]) {
                dfs(next, visited);
            }
        }
    }

    void dfsStart(int start) {

        boolean visited[] = new boolean[V];
```

```
        System.out.print("DFS: ");
        dfs(start, visited);
        System.out.println();
    }

    public static void main(String[] args) {

        Graph g = new Graph(6);

        g.addEdge(0, 2);
        g.addEdge(1, 4);
        g.addEdge(1, 5);
        g.addEdge(3, 5);
        g.addEdge(4, 2);
        g.addEdge(4, 5);

        g.bfs(0);
        g.dfsStart(1);
    }
}
```

Output:

```
PS C:\Users\HP> & 'C:\Program Files\jdk-17\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp '
'C:\Users\HP\AppData\Local\Temp\cp_aed4d455b175e91712e4501741e4a525\redhat.java\
jdt_ws\dynamicProgramming_43c6121c\bin' BFS_DFS'
BFS Order: 0 2
DFS Order: 1 4 2 5
PS C:\Users\HP>
```

12. N-Queen

Aim:

To place queens safely using backtracking.

Theory:

Backtracking avoids conflicts in rows, columns, diagonals.

Code :

```
public class NQueen {

    int N = 4;

    // check if safe
    boolean isSafe(int board[][], int row, int col) {

        // check left side
        for (int i = 0; i < col; i++) {
            if (board[row][i] == 1)
                return false;
        }

        // upper diagonal
        for (int i = row, j = col; i >= 0 && j >= 0; i--, j--) {
            if (board[i][j] == 1)
                return false;
        }

        // lower diagonal
        for (int i = row, j = col; i < N && j >= 0; i++, j--) {
            if (board[i][j] == 1)
                return false;
        }

        return true;
    }

    // solve using backtracking
    boolean solveQueen(int board[][], int col) {
```

```
        if (col >= N)
            return true;

        for (int i = 0; i < N; i++) {

            if (isSafe(board, i, col)) {

                board[i][col] = 1;

                if (solveQueen(board, col + 1))
                    return true;

                // backtrack
                board[i][col] = 0;
            }
        }

        return false;
    }

    void solve() {

        int board[][] = new int[N][N];

        if (!solveQueen(board, 0)) {
            System.out.println("No solution");
            return;
        }

        print(board);
    }

    void print(int board[][]) {

        for (int i = 0; i < N; i++) {

            for (int j = 0; j < N; j++) {
                System.out.print(board[i][j] + " ");
            }

            System.out.println();
        }
```

```
    }

    public static void main(String[] args) {

        NQueen obj = new NQueen();
        obj.solve();
    }
}
```

Output:

```
PS C:\Users\HP>  &
in' 'NQueen'
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
PS C:\Users\HP>
```

13. Sum Of Subsets

Aim:

To find subsets with given sum.

Theory:

Uses backtracking with include/exclude approach.

Code:

```
public class SumOfSubsets {

    int arr[] = {5, 11, 12, 13, 16, 18};
    int target = 30;
    int n = arr.length;

    void find(int sum, int index, boolean selected[]) {

        // if target reached
        if (sum == target) {

            System.out.print("Subset: ");

            for (int i = 0; i < n; i++) {
                if (selected[i]) {
                    System.out.print(arr[i] + " ");
                }
            }

            System.out.println();
            return;
        }

        // stop condition
        if (sum > target || index == n) {
            return;
        }

        // include current element
        selected[index] = true;
```

```
        find(sum + arr[index], index + 1, selected);

        // exclude current element
        selected[index] = false;
        find(sum, index + 1, selected);
    }

    public static void main(String[] args) {

        SumOfSubsets obj = new SumOfSubsets();

        boolean selected[] = new boolean[obj.n];

        obj.find(0, 0, selected);
    }
}
```

Output:

```
PS C:\Users\HP> & 'C:\Program Files\jdk-17\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
'C:\Users\HP\AppData\Local\Temp\cp_aed4d455b175e91712e4501741e4a525\redhat.java\
jdt_ws\dynamicProgramming_43c6121c\bin' 'SumOfSubsets'
Subset: 5 12 13
Subset: 12 18
PS C:\Users\HP>
```

14. Naive String Matching

Aim:

To find pattern occurrences in text.

Theory:

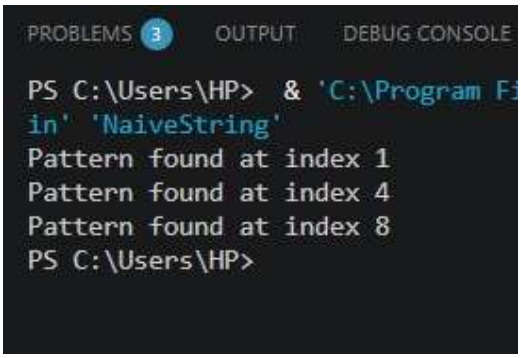
Brute-force method checks all positions.

Code:

```
public class NaiveString {

    static void search(String text, String pattern) {

        int n = text.length();
        int m = pattern.length();

        for (int i = 0; i <= n - m; i++) {

            int j;

            for (j = 0; j < m; j++) {

                if (text.charAt(i + j) != pattern.charAt(j)) {
                    break;
                }
            }

            if (j == m) {
                System.out.println("Pattern found at index " + i);
            }
        }
    }

    public static void main(String[] args) {

        String text = "112312311231124";
        String pattern = "1231";

        search(text, pattern);
```

```
    }
}
```

Output:



15. Floyd warshall

Aim:

To find shortest path between all pairs.

Theory:

Dynamic programming approach for all-pairs shortest path.

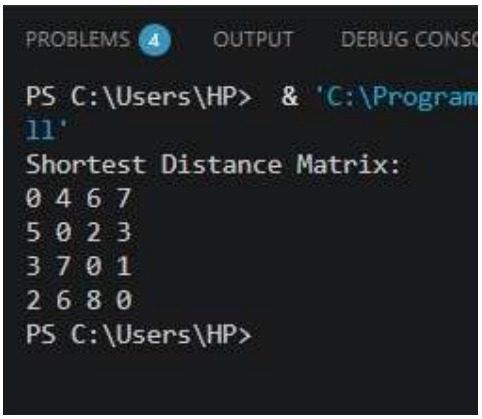
Code:

```
public class FloydWarshall {

    static int INF = 9999;

    static void solve(int graph[][]) {

        int n = graph.length;

        int dist[][] = new int[n][n];
```

```
        // copy graph
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                dist[i][j] = graph[i][j];
            }
        }

        // main logic
        for (int k = 0; k < n; k++) {

            for (int i = 0; i < n; i++) {

                for (int j = 0; j < n; j++) {

                    if (dist[i][k] != INF && dist[k][j] != INF) {

                        if (dist[i][k] + dist[k][j] < dist[i][j]) {
                            dist[i][j] = dist[i][k] + dist[k][j];
                        }
                    }
                }
            }
        }

        // print result
        System.out.println("Shortest Path Matrix:");

        for (int i = 0; i < n; i++) {

            for (int j = 0; j < n; j++) {

                if (dist[i][j] == INF)
                    System.out.print("INF ");
                else
                    System.out.print(dist[i][j] + " ");
            }

            System.out.println();
        }
    }

    public static void main(String[] args) {
```

```
        int graph[][] = {
            {0, 4, INF, 7},
            {8, 0, 2, INF},
            {5, INF, 0, 1},
            {2, INF, INF, 0}
        };

        solve(graph);
    }
}
```

Output:



16. Activity Selection

Aim:

To select the maximum number of non-overlapping activities using greedy approach.

Theory:

Activity Selection is a greedy algorithm problem where we select activities based on their finishing time. The activity that finishes earliest is selected first to maximize the number of activities.

Code:

```
import java.util.*;

class Activity {
    int start, finish;

    Activity(int s, int f) {
        start = s;
        finish = f;
    }
}

public class ActivitySelection {

    static void solve(Activity arr[]) {

        // sort by finish time
        Arrays.sort(arr, (a, b) -> a.finish - b.finish);

        System.out.println("Selected Activities:");

        // first activity always selected
        int lastFinish = arr[0].finish;
        System.out.println("(" + arr[0].start + ", " + arr[0].finish +
")");

        // check remaining
        for (int i = 1; i < arr.length; i++) {

            if (arr[i].start >= lastFinish) {
```

```
                System.out.println("(" + arr[i].start + ", " +
arr[i].finish + ")");
                lastFinish = arr[i].finish;
            }
        }
    }

    public static void main(String[] args) {

        Activity arr[] = {
            new Activity(3, 6),
            new Activity(2, 4),
            new Activity(5, 6),
            new Activity(7, 9),
            new Activity(6, 10),
            new Activity(8, 11)
        };

        solve(arr);
    }
}
```

Output: